

INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA

Ingeniería de Software y Datos

Documentación de la Arquitectura y Modelo de Datos

Proyecto integrador de Big Data: ingesta, preprocesamiento y
enriquecimiento en una nube simulada

Estudiante	Jhon Jairo Fuentes Turizo
Curso	Infraestructura y arquitectura para Big Data (PREICA2602B010136)
Actividad	EA4. Documentación de la Arquitectura y Modelo de Datos
Docente	César Luis Vásquez
Repositorio	github.com/Jhon-Fuenwhv2024/ea1_infraestructura
Fecha	septiembre de 2026

Contenido

1. Introducción y alcance.....	3
2. Visión global de la arquitectura.....	3
3. Componentes del sistema	4
4. Diagramas de flujo	5
5. Modelo de datos	6
6. Justificación tecnológica	8
7. Simulación de la nube, flujo y automatización.....	9
8. Conclusiones, limitaciones y recomendaciones	10
Referencias	11

Resumen. Este documento describe la arquitectura y el modelo de datos del proyecto integrador de Big Data construido a lo largo del curso: una ingesta desde la API pública Open Brewery DB (Actividad 1), el almacenamiento en una base SQLite que simula la nube (`src/static/db/ingestion.db`) y un preprocesamiento con Pandas automatizado con GitHub Actions (Actividad 2). Todas las cifras del texto se calcularon directamente sobre los artefactos del repositorio (base de datos, hojas de cálculo, informes de auditoría y código fuente); las secciones de límites señalan expresamente lo que el repositorio no contiene.

1. Introducción y alcance

El proyecto integrador del curso busca el ciclo de datos de una plataforma de Big Data: extraer datos desde una fuente externa, almacenarlos, depurarlos y prepararlos para etapas posteriores de enriquecimiento y modelado. Se emplea una nube comercial; el almacenamiento en la nube se **simula** con una base de datos local y con scripts versionados en GitHub, cuya ejecución se automatiza con GitHub Actions.

El objetivo de este documento es dejar constancia técnica de cómo está construido ese sistema: qué componentes lo forman, cómo fluyen los datos entre ellos, cuál es el modelo de datos real y por qué se tomaron las decisiones tecnológicas. Para evitar inconsistencias entre la documentación y el código, el modelo de datos se leyó de la propia base de datos (`PRAGMA table_info` y `PRAGMA foreign_key_list`) y los diagramas de flujo se derivaron de las funciones de `src/ingestion.py` y `src/cleaning.py`.

Trazabilidad del trabajo previo

Actividad	Entregable en el repositorio	Evidencia verificada
EA1 · Ingesta desde API	<code>src/ingestion.py</code> , <code>src/static/auditoria/ingestion.txt</code> , <code>src/static/xlsx/ingestion.xlsx</code>	Auditoría del 30-ago-2026: 11 848 registros extraídos de la API y 11 848 almacenados en MySQL; sin discrepancias en campos clave.
EA2 · Preprocesamiento	<code>src/cleaning.py</code> , <code>src/static/db/ingestion.db</code> , <code>src/static/xlsx/cleaned_data.xlsx</code> , <code>src/static/auditoria/cleaning_report.txt</code>	3 025 filas de entrada → 3 000 filas limpias (5 duplicados y 20 filas basura eliminadas); workflow de GitHub Actions en verde el 16-sep-2026.
EA3 · Enriquecimiento	script de enriquecimiento en el repositorio.	Se documenta como etapa posterior de la arquitectura (sección 2); no se atribuyen resultados que no se puedan comprobar.
EA4 · Documentación	<code>docs/arquitectura_modelo.pdf</code> , <code>src/validate_model.py</code>	Este documento y el informe <code>schema_report.txt</code> generado desde la base de datos.

Fuente de los datos

Los datos provienen de Open Brewery DB, una API REST pública, sin autenticación, que devuelve en formato JSON el directorio de cervecerías con campos como tipo de cervecería, dirección, ciudad, país, coordenadas, teléfono y sitio web (Open Brewery DB, s. f.). La ingesta paginó la API con `per_page=200` hasta reunir 11 848 registros únicos.

2. Visión global de la arquitectura

La arquitectura sigue un patrón por etapas (*pipeline*) en el que cada etapa lee la salida de la anterior y deja evidencias auditables. La Figura 1 muestra los cuatro bloques: la fuente externa, el almacenamiento simulado en la nube, las etapas de procesamiento y la capa de automatización.

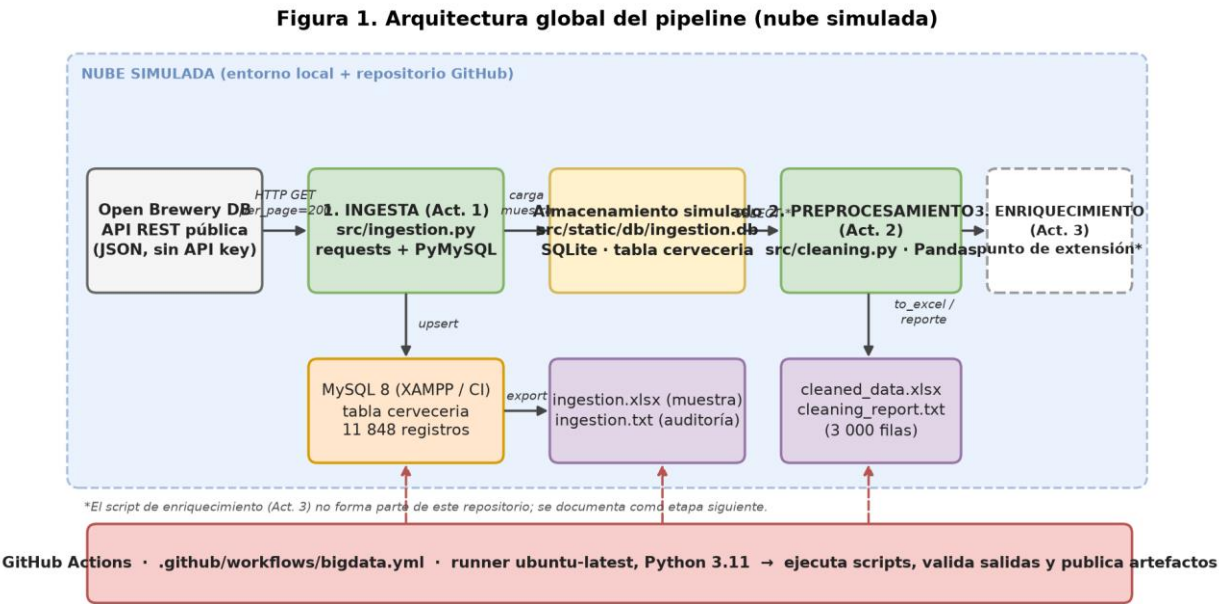


Figura 1. Arquitectura global del pipeline. Elaboración propia con matplotlib, a partir del código y los artefactos del repositorio.

Capa	Responsabilidad	Implementación en el proyecto
Fuente	Proveer los datos crudos.	API REST Open Brewery DB (JSON).
Ingesta	Extraer, deduplicar y cargar en la base; auditar la carga.	src/ingestion.py (requests, PyMySQL, pandas); MySQL 8 en XAMPP o en un servicio del runner.
Almacenamiento (nube simulada)	Conservar los datos en un repositorio central consultable con SQL.	Archivo SQLite src/static/db/ingestion.db, versionado en GitHub.
Preprocesamiento	Depurar y tipar los datos; medir el efecto.	src/cleaning.py (pandas, openpyxl) → cleaned_data.xlsx y cleaning_report.txt.
Enriquecimiento	Combinar el dataset limpio con fuentes externas por una clave común.	Punto de extensión: no está implementado en este repositorio (ver limitaciones).
Automatización	Ejecutar y validar el pipeline de forma repetible.	GitHub Actions: .github/workflows/bigdata.yml.

3. Componentes del sistema

Componente	Archivo	Función y detalles verificados en el código
Base de datos SQLite	src/static/db/ingestion.db	Una tabla, cervceria, con 14 columnas y 3 025 filas (3 000 registros reales de la API más 25 filas sintéticas sucias creadas a propósito para practicar la limpieza). PRAGMA integrity_check: ok.
Script de ingesta	src/ingestion.py	Funciones fetch_cervcerias, create_table, upsert_records (INSERT ... ON DUPLICATE KEY UPDATE, idempotente), export_xlsx (muestra de 500 filas), write_audit y dump_sql (mysqldump).
Script de limpieza y preprocesamiento	src/cleaning.py	Carga la tabla con pandas.read_sql_query, explora nulos y duplicados, aplica seis transformaciones (sección 4.2) y escribe el Excel limpio y el informe de auditoría.
Script de validación del modelo	src/validate_model.py	Lee el esquema con PRAGMA, comprueba tabla, columnas y PK, y genera schema_report.txt (agregado en esta actividad).
Evidencias	src/static/auditoria/, src/static/xlsx/	Informes de texto (ingestión, limpieza, esquema) y hojas de Excel (muestra cruda de 500 filas y dataset limpio de 3 000).

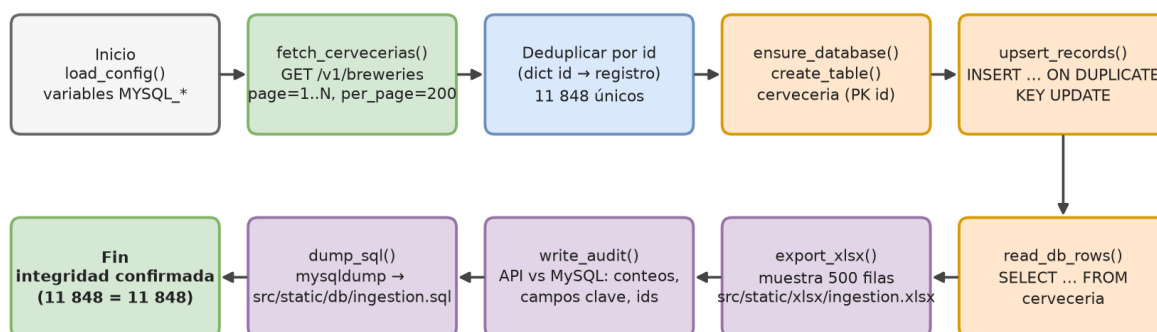
Componente	Archivo	Función y detalles verificados en el código
Automatización	.github/workflows/bigdata.yml	Tres jobs en ubuntu-latest con Python 3.11: <i>cleaning</i> , <i>validar-modelo</i> y <i>documentacion</i> ; publican artefactos con actions/upload-artifact@v4.
Empaquetado	setup.py, requirements.txt	Dependencias mínimas (pandas, openpyxl) y extras opcionales para la ingesta y la documentación.

4. Diagramas de flujo

4.1 Flujo API → base de datos (ingesta)

La Figura 2 describe la Actividad 1. El script lee la configuración, recorre las páginas de la API, elimina repetidos por `id`, crea la tabla si no existe y realiza un *upsert*. Después relea la tabla para exportar la muestra y para comparar API contra base de datos en el informe de auditoría.

Figura 2. Diagrama de flujo: API → base de datos (Act. 1, src/ingestion.py)



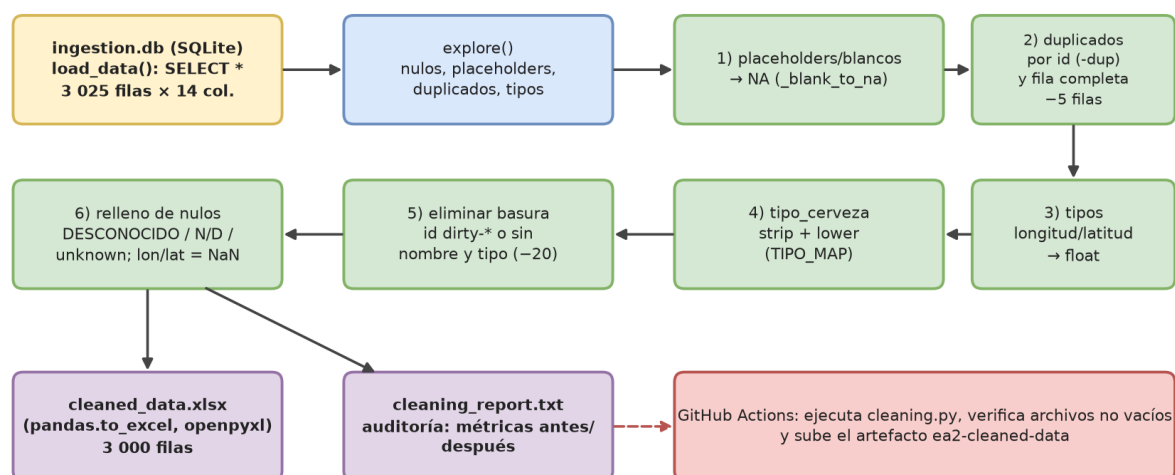
Fuente: `src/ingestion.py` del repositorio y `src/static/auditoria/ingestion.txt` (corrida del 30-ago-2026).

Figura 2. Flujo de la ingesta (`src/ingestion.py`). Elaboración propia.

4.2 Flujo base de datos → preprocesamiento

La Figura 3 detalla la Actividad 2. Las transformaciones se aplican en el orden mostrado; cada una deja su efecto cuantificado en `cleaning_report.txt`:

- **Nulos:** cadenas vacías y marcadores (*None*, *nan*, *null*, *n/a*) pasan a nulo real; luego los campos de texto se completan con *DESCONOCIDO*, *N/D* o *unknown*, y las coordenadas inválidas permanecen como NaN.
- **Duplicados:** se eliminan por `id` (ignorando el sufijo *-dup*) y por fila completa: 5 filas.
- **Tipos:** `longitud` y `latitud` pasan de texto a *float* con `pd.to_numeric(errors='coerce')`.
- **Normalización:** `tipo_cerveza` en minúsculas y sin espacios.
- **Filas basura:** se descartan las de `id dirty-*` o sin nombre ni tipo: 20 filas.

Figura 3. Diagrama de flujo: BD → preprocesamiento (Act. 2, src/cleaning.py)

Fuente: src/cleaning.py y src/static/auditoria/cleaning_report.txt (workflow verde el 16-sep-2026).

Figura 3. Flujo del preprocesamiento (src/cleaning.py). Elaboración propia.

5. Modelo de datos

5.1 Modelo físico real

La base `ingestion.db` contiene **una sola tabla**, `cerveceria`, con clave primaria simple `id` y sin claves foráneas declaradas (`PRAGMA foreign_key_list` devuelve cero filas). Todas las columnas están declaradas como `TEXT`; esto incluye longitud y latitud, cuya conversión numérica se realiza recién en la etapa de limpieza. La Tabla 1 es el diccionario de datos; la columna *Origen* indica el campo de la API, según la documentación de Open Brewery DB (s. f.), y el correspondiente en MySQL.

#	Columna	Tipo en SQLite	PK	Origen (API → MySQL)	Descripción	Tipo tras EA2
1	id	TEXT	Sí	id → id	Identificador único (UUID) de la cervecería.	texto
2	nombre	TEXT		name → nombre	Nombre de la cervecería.	texto
3	tipo_cerveza	TEXT		brewery_type → tipo_cerveza	Categoría (micro, brewpub, closed...).	texto (13 valores)
4	direccion	TEXT		address_1 → direccion_1	Dirección principal.	texto
5	ciudad	TEXT		city → ciudad	Ciudad.	texto
6	estado_provincia	TEXT		state_province → estado_provincia	Estado o provincia.	texto
7	codigo_postal	TEXT		postal_code → codigo_postal	Código postal (texto: conserva ceros y guiones).	texto
8	pais	TEXT		country → pais	País.	texto
9	longitud	TEXT		longitude → longitud	Coordenada; en la base incluye vacíos y valores inválidos.	float64
10	latitud	TEXT		latitude → latitud	Coordenada; ídem.	float64
11	telefono	TEXT		phone → telefono	Teléfono de contacto.	texto
12	sitio_web	TEXT		website url → sitio_web	URL del sitio.	texto
13	estado	TEXT		state → estado	Campo <i>deprecated</i> de la API; redundante.	texto

#	Columna	Tipo en SQLite	PK	Origen (API → MySQL)	Descripción	Tipo tras EA2
14	street	TEXT		street → direccion_4	Campo <i>deprecated</i> ; redundante.	texto

Tabla 1. Diccionario de datos de la tabla *cerveceria* (fuente: PRAGMA table_info sobre src/static/db/ingestion.db).

Las columnas *direccion* y *street* del SQLite corresponden a *direccion_1* y *direccion_4* del esquema MySQL; la equivalencia se comprobó cruzando por *id* las 113 filas de la muestra *ingestion.xlsx* que están presentes en la base (coincidencia del 100 %). Las columnas *direccion_2*, *direccion_3* e *ingerido_en* de MySQL no se trasladaron al SQLite.

Figura 4. Diagrama entidad-relación del modelo físico real (notación de tabla)

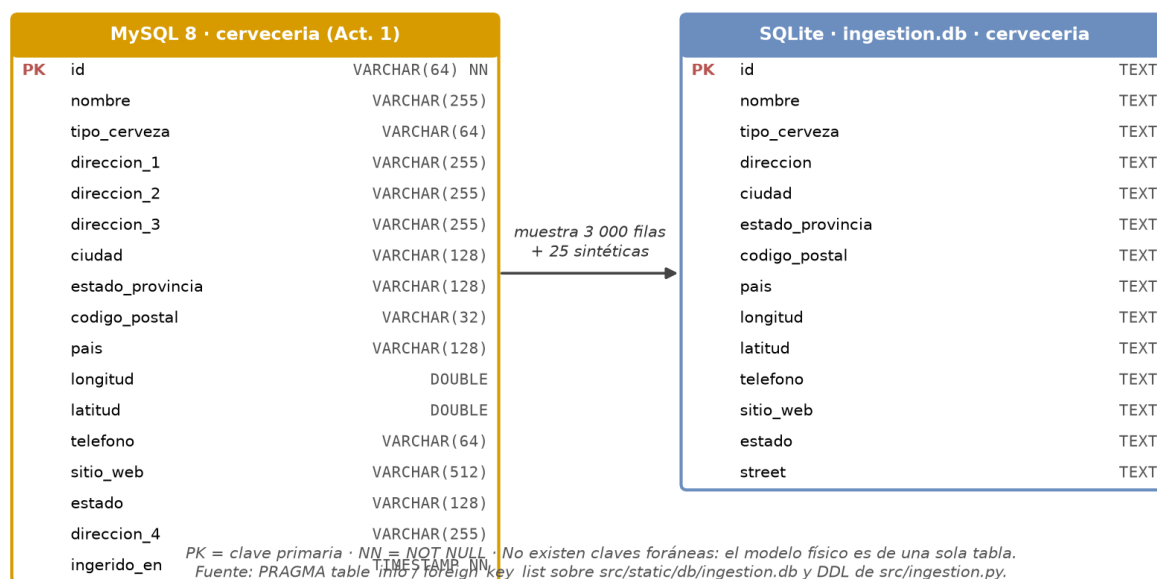


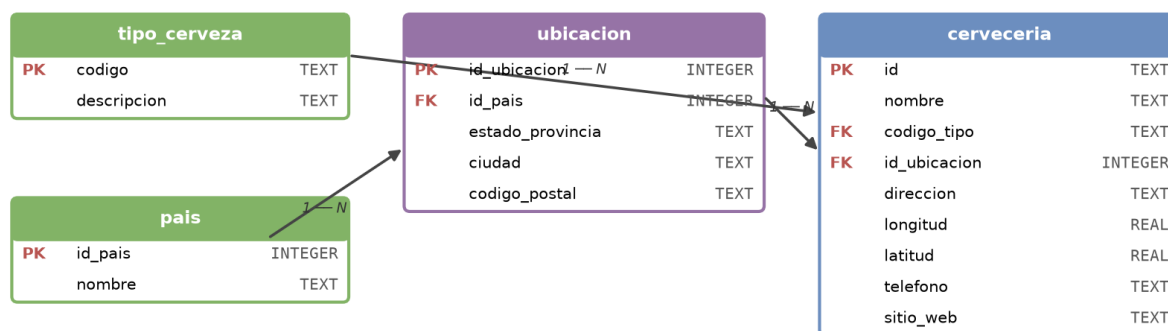
Figura 4. Modelo físico real: tabla MySQL de la Actividad 1 y tabla SQLite usada desde la Actividad 2.

5.2 Justificación del diseño actual

- **Tabla única y plana:** el propósito es analítico y de lectura por lotes; un solo conjunto rectangular se carga directamente en un *DataFrame* sin uniones. Cada fila describe una única entidad (una cervecería).
- **Clave primaria *id*:** el identificador proviene de la fuente, es estable e impide duplicados exactos; es la clave que permitirá enriquecer el dataset mediante uniones.
- **Tipo TEXT en todas las columnas:** SQLite usa afinidad de tipos y admite valores vacíos o inválidos sin rechazar la carga, lo que permite auditar los defectos de calidad en la limpieza en lugar de ocultarlos en la ingesta.
- **Cardinalidades observadas en el dataset limpio:** 22 países, 13 tipos de cervecería, 154 estados/provincias y 1874 ciudades distintas, sobre 3 000 registros.

5.3 Propuesta de evolución: modelo lógico normalizado

La tabla plana repite valores como país, estado y tipo en miles de filas, lo que en una base transaccional generaría anomalías de actualización. Siguiendo el modelo relacional (Codd, 1970) y el enfoque entidad-relación (Chen, 1976), la Figura 5 propone separar catálogos y ubicación (tercera forma normal). **Es una propuesta de diseño; no está implementada en el repositorio**, donde se conserva el modelo de tabla única descrito arriba.

Figura 5. Modelo lógico normalizado (PROPUESTA de evolución, no implementada)

Relaciones: tipo_cerveza 1:N cerveza · ubicacion 1:N cerveza · pais 1:N ubicacion.

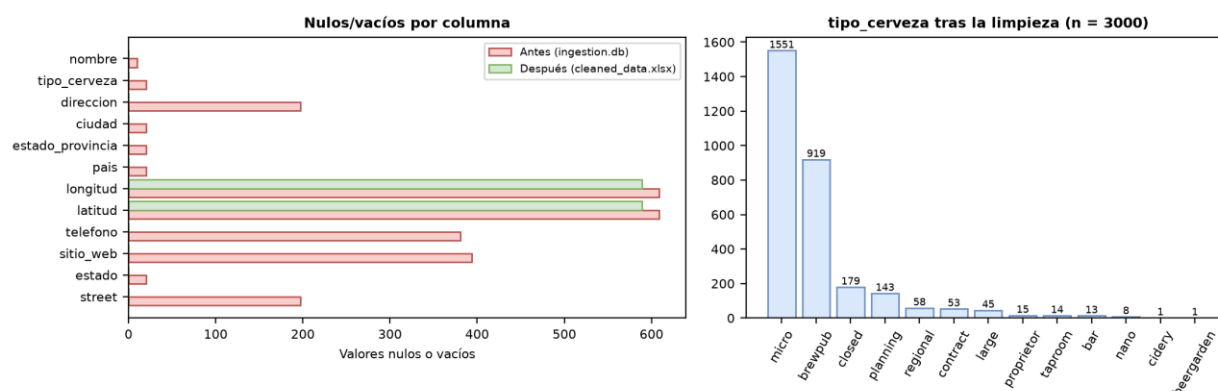
Es una propuesta de diseño (forma normal 3FN) derivada de los campos reales; el repositorio usa la tabla única cerveza.

Figura 5. Modelo lógico normalizado propuesto (relaciones 1:N). Derivado de los campos reales.

Relación propuesta	Cardinalidad	Justificación
tipo_cerveza → cerveza	1 : N	Un tipo agrupa muchas cervecerías; centraliza la escritura del catálogo (p. ej. «micro»).
pais → ubicacion	1 : N	Un país contiene muchas ubicaciones; evita repetir el nombre del país.
ubicacion → cerveza	1 : N	Una ubicación (estado, ciudad, código postal) puede alojar varias cervecerías.

5.4 Efecto del preprocesamiento sobre el modelo

La Figura 7 compara los nulos o vacíos por columna antes y después de la limpieza, calculados sobre la base y sobre el Excel limpio. Tras la limpieza no quedan nulos en las columnas de texto, pero **589 filas (19.6%) conservan longitud y latitud vacías**, porque el dato no existe en la fuente y no se imputa.

Figura 7. Efecto medido del preprocesamiento (calculado desde los artefactos del repositorio)**Figura 7. Calidad antes y después de la limpieza (calculado desde ingestion.db y cleaned_data.xlsx).**

6. Justificación tecnológica

Tecnología	Rol en el proyecto	Por qué se eligió	Límites a tener en cuenta
------------	--------------------	-------------------	---------------------------

Tecnología	Rol en el proyecto	Por qué se eligió	Límites a tener en cuenta
SQLite	Base que simula el almacenamiento en la nube (<i>ingestion.db</i>).	No requiere servidor ni administración, es un único archivo portable y versionable en Git, y el runner de Actions lo lee sin servicios adicionales. Sus propios autores lo recomiendan para almacenamiento local y formatos de archivo de aplicaciones (SQLite, s. f.).	Un solo escritor a la vez; no es adecuado para muchos clientes concurrentes por red ni para conjuntos de datos muy grandes (SQLite, s. f.).
Pandas	Lectura SQL, limpieza, tipado y exportación a Excel.	Estructura tabular en memoria (<i>DataFrame</i>) con operaciones vectorizadas de nulos duplicados y conversión de tipos; integra <i>read_sql_query</i> y <i>to_excel</i> (McKinney 2010). Los 3 025 registros caben holgadamente en memoria.	Procesa en un solo nodo; el volumen debe caber en RAM.
PySpark	No se utiliza en la implementación actual. Se propone como vía de escalamiento.	PySpark es la API de Python de Apache Spark, un motor unificado para procesamiento por lotes, en flujo e interactivo (Apache Spark, s. f.; Zaharia et al., 2016). Permitiría reescribir <i>cleaning.py</i> con <i>DataFrames</i> distribuidos si el volumen superara la memoria de un nodo.	Añade una JVM y sobrecarga de arranque que no se justifican con miles de filas.
GitHub Actions	Automatización de la limpieza, la validación del modelo y la verificación de la documentación.	Es una plataforma de CI/CD integrada en el repositorio que ejecuta flujos de trabajo ante <i>push</i> , <i>pull request</i> o ejecución manual (GitHub, s. f.); deja artefactos descargables como evidencia.	Los runners son efímeros: los archivos generados se descargan como artefactos y no se confirman solos en el repositorio.

7. Simulación de la nube, flujo y automatización

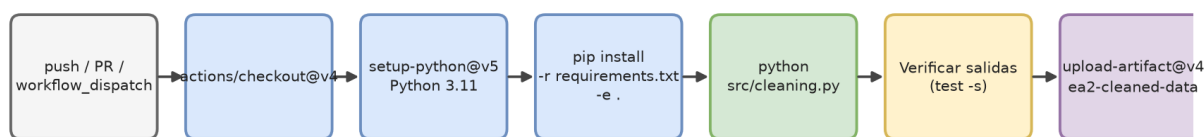
7.1 Cómo se simula la nube

Una plataforma de Big Data en la nube ofrece almacenamiento centralizado, cómputo bajo demanda y ejecución automatizada. El proyecto reproduce cada rol a pequeña escala:

Servicio en la nube real	Equivalente simulado en el proyecto
Almacenamiento / <i>data lake</i> o base gestionada	Archivo SQLite <i>ingestion.db</i> versionado en GitHub; tabla <i>cerveceria</i> .
Servicio de ingesta	<i>src/ingestion.py</i> (Act. 1): API → MySQL, con auditoría de integridad.
Cómputo de transformación	<i>src/cleaning.py</i> (Act. 2) sobre pandas, ejecutado en el runner de GitHub.
Enriquecimiento	Etapla prevista: uniones por <i>id</i> o por país/ciudad con fuentes externas; no implementada aquí.
Orquestación y monitoreo	GitHub Actions con verificación de salidas y artefactos.
Catálogo y auditoría	Archivos de <i>src/static/auditoria/</i> (<i>ingestion.txt</i> , <i>cleaning_report.txt</i> , <i>schema_report.txt</i>).

7.2 Flujo de automatización

El workflow *bigdata.yml* se dispara con *push* a *main*, *master* o *ea4-documentacion*, con *pull request* y manualmente. La Figura 6 muestra el job *cleaning*; los jobs *validar-modelo* y *documentacion* son independientes y se ejecutan en paralelo.

Figura 6. Flujo de automatización: job «cleaning» de GitHub Actions (bigdata.yml)

El mismo workflow define otros dos jobs independientes: «validar-modelo» (comprueba esquema/PK de ingestion.db y genera schema_report.txt) y «documentacion» (verifica la estructura requerida y publica el PDF como artefacto).

Figura 6. Job «cleaning» del workflow de GitHub Actions.

- **cleaning:** instala dependencias, ejecuta `src/cleaning.py`, comprueba que el Excel y el informe existan y no estén vacíos, y publica el artefacto `ea2-cleaned-data`.
- **validar-modelo:** ejecuta `src/validate_model.py`, que falla si cambian la tabla, las columnas o la PK, y publica `ea4-schema-report`.
- **documentacion:** verifica que existan `setup.py`, `README.md`, el workflow, la base de datos y este PDF, y publica `ea4-arquitectura-pdf`.

Historial verificado en GitHub: la versión anterior del workflow (EA2) terminó con éxito el 16 de septiembre de 2026 (commit `b0f53b`). El workflow ampliado de esta actividad se validó localmente (sintaxis YAML y ejecución de los scripts), pero su primera ejecución en Actions ocurrirá al hacer *push* del repositorio.

8. Conclusiones, limitaciones y recomendaciones

8.1 Conclusiones

- La arquitectura por etapas (fuente → ingesta → almacenamiento → preprocesamiento → automatización) es simple, reproducible y auditable: cada etapa deja evidencias en `src/static/`.
- El preprocesamiento redujo la base de 3 025 a 3 000 filas sin duplicados, tipó las coordenadas y eliminó los nulos de los campos de texto, con un informe que cuantifica cada cambio.
- El modelo de tabla única con PK natural es suficiente para el análisis actual y, gracias a la clave *id*, admite integrarse con otras fuentes en una etapa de enriquecimiento.
- SQLite + Pandas + GitHub Actions ofrecen un entorno de bajo costo y sin servidores que representa, a escala reducida, los roles de una plataforma de Big Data en la nube.

8.2 Limitaciones

- **Volumen:** la base contiene 3 000 registros reales (25,3 % de los 11 848 ingeridos en MySQL) y 25 filas sintéticas; no es un escenario de Big Data en sentido estricto.
- **Nube simulada:** no hay servicios gestionados, escalado ni tolerancia a fallos.
- **Modelo sin restricciones:** una sola tabla, sin claves foráneas y con todas las columnas en TEXT; no hay integridad referencial ni validación de tipos en la base.
- **Trazabilidad incompleta:** el repositorio no incluye el script que trasladó la muestra de MySQL a SQLite ni las 25 filas sintéticas; tampoco incluye el script de enriquecimiento (Act. 3).
- **PySpark:** se justifica conceptualmente, pero no se ejecutó en el proyecto.
- **Calidad residual:** 589 registros sin coordenadas; los marcadores *N/D*, *DESCONOCIDO* y *unknown* mezclan un centinela de texto con datos válidos; los tipos *taproom*, *cidery* y *beergarden* aparecen en

los datos aunque no figuran en la lista de valores de la documentación consultada de la API (Open Brewery DB, s. f.).

- **Automatización:** los artefactos de Actions no se confirman en el repositorio; las copias versionadas se regeneran con los mismos scripts.

8.3 Recomendaciones

- Versionar el script de carga MySQL → SQLite y un script *enrichment.py* que una el dataset limpio con fuentes adicionales por *id* o por país, documentando registros coincidentes en un informe de auditoría.
- Migrar a un modelo normalizado (Figura 5) con claves foráneas activadas (`PRAGMA foreign_keys = ON`) y tipos *REAL* para las coordenadas; sustituir los centinelas de texto por nulos y validar los tipos en la carga.
- Ingerir el conjunto completo de 11 848 registros y, si el volumen o el número de fuentes crece, reescribir la limpieza con PySpark y almacenar en un sistema de archivos distribuido o en un almacén columnar.
- Agregar pruebas automáticas (por ejemplo, con *pytest*) de las reglas de limpieza y ejecutarlas en el workflow.
- No versionar credenciales: mantener `.env` fuera del repositorio y publicar solo `.env.example`.

Referencias

Apache Spark. (s. f.). *PySpark overview*. Apache Software Foundation. Recuperado el 29 de septiembre de 2026, de <https://spark.apache.org/docs/latest/api/python/index.html>

Chen, P. P.-S. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9-36. <https://doi.org/10.1145/320434.320440>

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>

GitHub. (s. f.). *Understanding GitHub Actions*. GitHub Docs. Recuperado el 29 de septiembre de 2026, de <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt y J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56-61). <https://doi.org/10.25080/Majora-92bf1922-00a>

Open Brewery DB. (s. f.). *Documentation*. Recuperado el 29 de septiembre de 2026, de <https://openbrewerydb.org/documentation>

SQLite. (s. f.). *Appropriate uses for SQLite*. Recuperado el 29 de septiembre de 2026, de <https://www.sqlite.org/whentouse.html>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., y Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56-65. <https://doi.org/10.1145/2934664>